

[Sign In](#)[Free Startup Course](#)

How to Systematically Prioritize and Tackle the Riskiest Assumptions in Your Business Model

The GO-LEAN Framework



ASH MAURYA
CONTINUOUS INNOVATION

When launching a new product under extreme uncertainty, it's critical to prioritize tackling your riskiest assumptions first. While a simple enough concept to grasp, it's ironically quite challenging to put into

Subscribe to the Newsletter

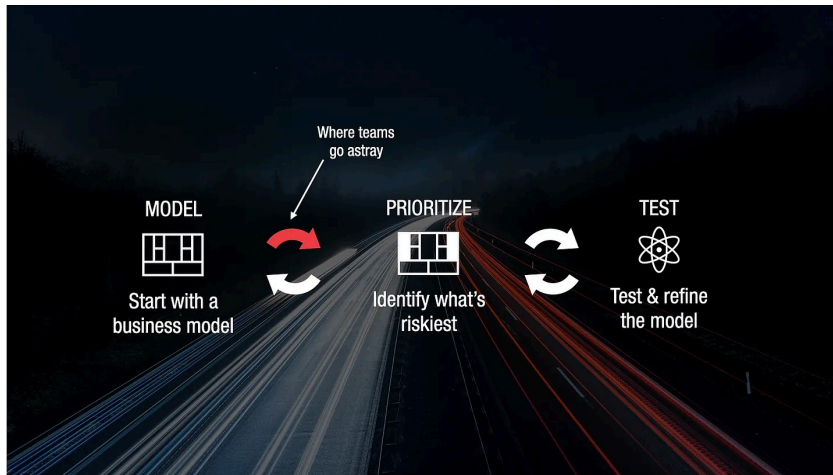
Join thousands of founders for battle-tested recipes, strategies, and how-tos for achieving product/market fit systematically.

Send
Me
Weekly
Startup
Tips →

SHARE THIS ARTICLE ON:



practice — and where lots of entrepreneurs go astray:



Too many entrepreneurs guess at their riskiest assumptions using their intuition or by seeking the advice of other “experts.” But when building something new that’s never been attempted, relying on the opinions of even so-called experts is prone to bias.

Voting on the most popular opinions either amplifies the bias of the voting group or averages it out.

Neither is good.

Incorrect prioritization of risk is one of the top contributors to waste in a startup.

Why is that? Because when you misdiagnose your riskiest assumption, you waste needless time, money, and effort focusing on the wrong

actions. This depletes your already limited resources and shortens your runway to find product/market fit.

I'll outline a framework for systematically surfacing your riskiest assumptions and sidestepping these pitfalls in today's issue.

Subscribed

Meet the GO LEAN Framework

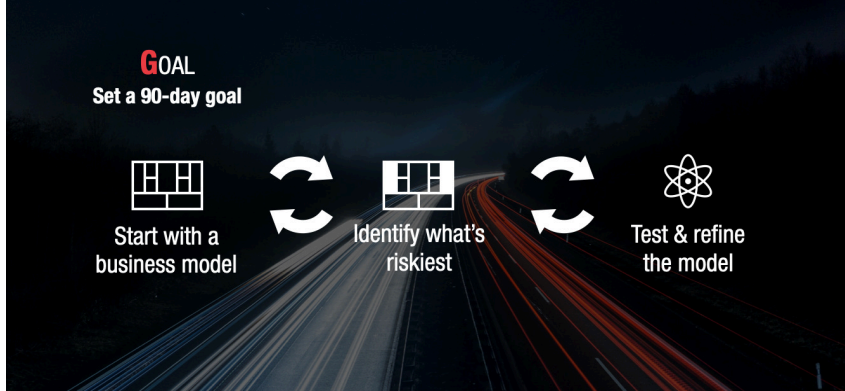
GO LEAN is a mnemonic I created to help you remember the steps:

1. **G**oal
2. **O**bstacle
3. **L**earn
4. **E**xperiment
5. **A**nalyze
6. **N**ext Action

Let's walk through them.

1. G(oal)

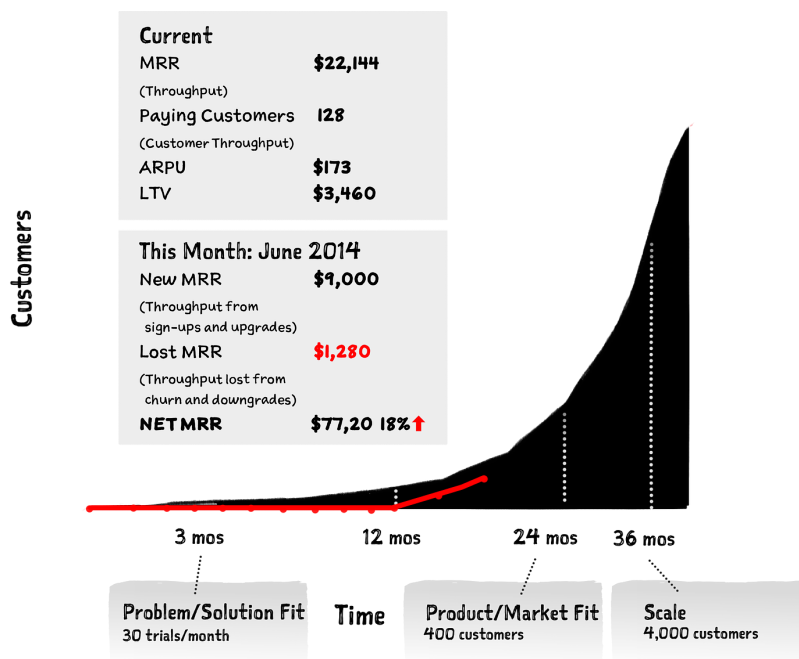
The first step to optimizing any journey is getting clear on your destination.



What's riskiest when constructing a two-story house differs from building a skyscraper.

While it's hard (if not impossible) to get precise on goals for an early-stage product, you can almost always identify a reasonable ballpark with a little effort.

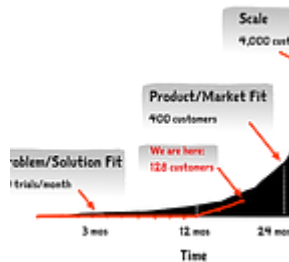
My favorite model for doing this is the Traction Roadmap.



A traction roadmap helps identify the key milestones in your journey from idea to scale.

More importantly, it lets you baseline your progress and extrapolate a 90-day goal.

Getting clear on your next 90-day goal is key to driving focus on the immediate obstacles in your way.



Don't Create a Product Roadmap. Use a Traction Roadmap Instead.

2. O(bstacle)

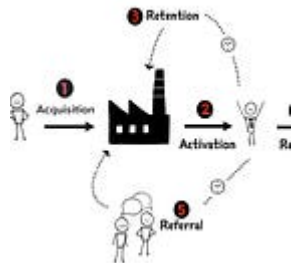
Obstacles or constraints hold the key to uncovering and challenging your riskiest assumptions.



When asked about obstacles, it's typical to list too many. But not all obstacles are equal when we think of a business as a system. In any system, only a single bottleneck or

constraint (weakest link) is always holding back throughput.

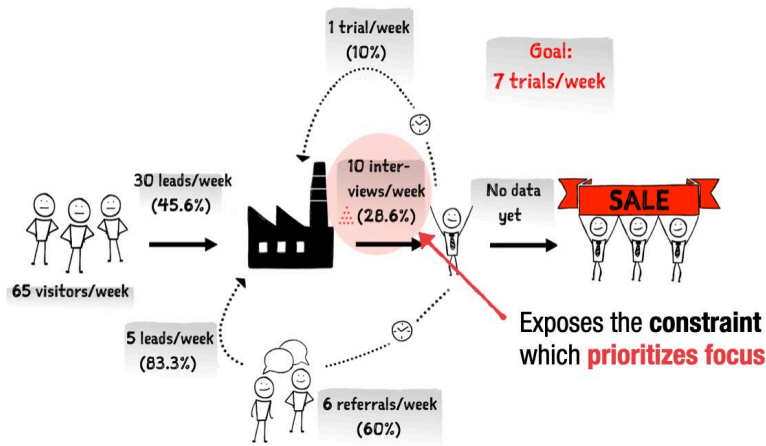
The mental model I use to identify the one obstacle or constraint is the Customer Factory coupled with the Theory of Constraints (TOC).



The Customer Factory Manifesto

Unlock your constraint, and you unlock throughput (traction).

A Customer Factory breaks the customer journey from unaware visitor to happy customer into five macro steps. When you map these steps into specific user actions that you start measuring weekly, bottlenecks and constraints become apparent.



Once an obstacle is identified, it's too easy to fall prey to guessing or, more accurately, wishing for more resources (more people, money, and time), which brings us to the next step.

3. L(earn)

Behind every obstacle are a set of root causes or insights that hold the key to breaking a constraint.



While most small teams typically aim to divide and conquer multiple obstacles in a business, you stand to drive the biggest impact by focusing your already limited resources on the single obstacle or constraint holding you back.

How you work around, rather than fall victim, to your obstacles separates success from failure.

This requires explorative discovery. For any given obstacle, there can be multiple possible solutions. But here, too, not all will be equally effective.

The steps to finding good solutions are

1. fostering a diversity of possible solutions from your team to avoid the curse of specialization,
2. requiring solution proposals to be accompanied by problem evidence — insights,
3. shortlisting possible solutions based on potential problem/solution fit strength — placing bets.

4. E(xperiment)

Every big solution (or campaign) can be broken into a series of small and fast additive experiments.



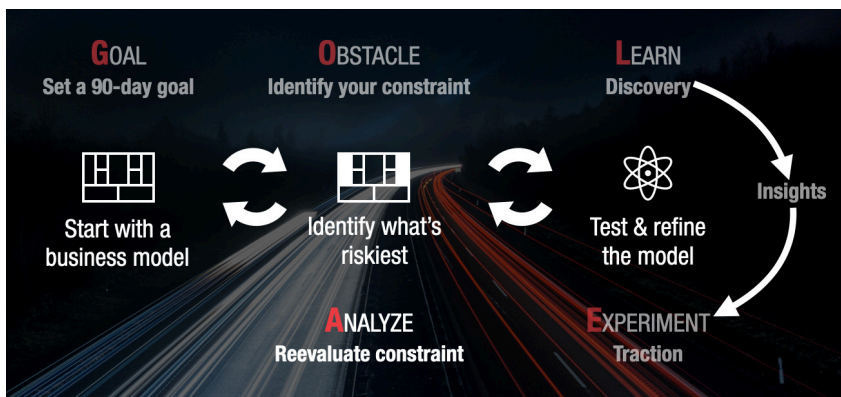
With possible solutions shortlisted, the next step is running a series of small and fast (time-boxed) additive experiments and putting the insights (or riskiest assumptions) to the test.

Good experiments prioritize the riskiest assumptions first.

When you run experiments this way, its easy to find signals in the noise and double down on your most promising insights.

5. A(nalyze)

Be wary of local optimization.



Constraints or obstacles in a system move around unpredictably. In other words, when a solution starts working well enough, it breaks the current constraint, and a new constraint emerges somewhere else in the Customer Factory.

Failing to detect that a constraint has been broken leads to over-optimization and diminishing returns.

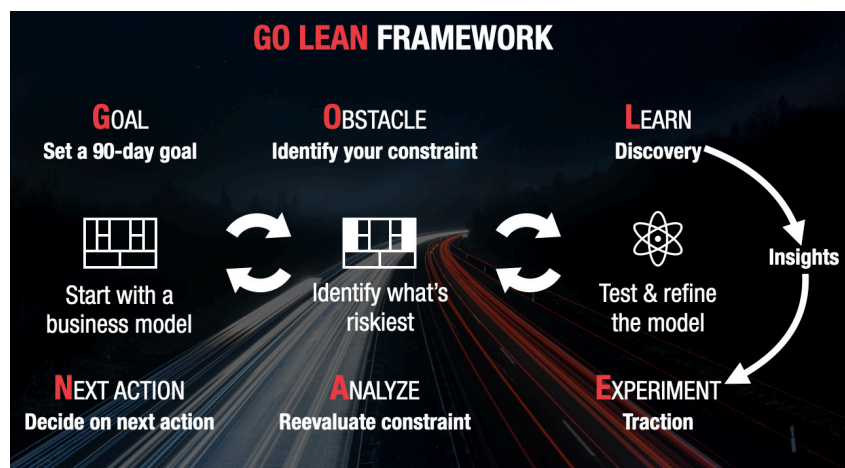
This is why it's critical to instill a metrics culture across your team where you review your customer factory throughput weekly and reassess constraints.

6. N(ext) Action

At any given time, there are only a few key actions that stand to drive the biggest impact. Focus on those and ignore the rest.

When you systematize prioritizing your riskiest assumptions as described above, each action follows the previous action — backed by goals, assumptions, constraints, learning, and results.

Step by step, you march forward.



Read Next:

The Customer Factory

Successful businesses are more alike than unlike. They share a common universal goal and employ a systematic approach to building a repeatable and scalable